
UkrAid

Global Solution 2026 - 1° Semestre



1. O Problema e a Solução

O contínuo conflito ocorrendo dentro da Ucrânia criou uma assimetria de informação: Os avisos são noticiados com atraso ou até mesmo tarde demais, podem ser imprecisos ou não levam em conta a localização específica de um civil. Basicamente uma pessoa a 50 metros de distância de um campo de batalha tem uma exposição ao risco fundamentalmente diferente de outra pessoa a 50 quilômetros de distância.

O UkrAid é uma CLI (Command Line Interface) desenvolvida como solução para a Global Solution do 1º Semestre de 2026. O sistema procura centralizar dados de inteligência de fontes abertas (OSINT) para auxiliar civis em zonas de conflito.

O modelo computacional aplica fórmulas de **atenuação exponencial** para traduzir a distância bruta do evento (calculada pela fórmula Haversine) em uma pontuação de risco baseada na distância, diferenciando entre ameaças móveis persistentes (movimentos de tropas) e eventos cinéticos agudos (ataques com mísseis).

O sistema de pontuação de risco permite que a solução forneça ao usuário um panorama situacional priorizado, abordando diretamente o problema humanitário da assimetria de informações em zonas de conflito ativo.

3. Objetivos do Sistema

- Calcular **pontuações de risco** baseadas na distância entre eventos de conflito e as coordenadas do usuário.
- Identificar e classificar os recursos humanitários mais próximos do usuário.
- Exibir o status em tempo real dos alertas de ataque aéreo para todas as regiões da Ucrânia, utilizando dados da *API Ukraine Alarm*.
- Disponibilizar uma interface de terminal limpa e orientada por menus, com validação de entrada em tempo real e saída formatada para facilitar o uso em situações de estresse.
- Operar de forma modular, com a integração das APIs, a modelagem de risco e as funções de suporte separadas em módulos distintos (`api.py`, `risk_model.py` e `support_functions.py`).

4. Principais Funções do Programa

4.1 Risco Baseado a Distância (Opção 2)

Este é o núcleo analítico do sistema. Após o usuário fornecer sua latitude e longitude, a aplicação:

- Chama a função **get_country()** para realizar a geocodificação reversa das coordenadas por meio do serviço **Overpass (OpenStreetMap)**.

- Obtém os 20 eventos de conflito mais recentes daquele país a partir da base de dados da **ACLED**, utilizando autenticação **OAuth2 Bearer Token**.
- Ordena todos os eventos pela distância geodésica em relação ao usuário utilizando a **fórmula de Haversine** e mantém apenas os 3 eventos mais próximos.
- Aplica uma função de risco específica para o tipo de evento (**troop_risk** ou **missile_strike**) e classifica o resultado por meio da função **risk_label()**.

As funções de risco utilizadas são:

```
#K_ Consts:
# Threat attenuation constants
K_TROOPS = 0.001      # persistent, mobile threat
K_MISSILE_MAX = 0.005 # highest risk per distance unit

def troop_risk(d, k = K_TROOPS):
    return math.exp(-k * d) / (d ** 2)

def missile_strike(d, k = K_MISSILE_MAX):
    return math.exp(-k * d) / (d ** 2)
```

Ambas seguem a mesma fórmula matemática:

$$R(d) = \frac{e^{-kd}}{d^2}$$

Essa equação combina uma **atenuação física inversamente proporcional ao quadrado da distância** com um **decaimento exponencial**.

A constante **k** assume um valor maior para eventos de ataque com mísseis (**0,005**) do que para movimentações de tropas (**0,001**), refletindo o fato de que os efeitos destrutivos de explosões tendem a diminuir mais rapidamente com a distância, enquanto a ameaça representada por forças militares móveis pode permanecer relevante por uma área geográfica mais ampla e por períodos mais longos.

4.2 Localizador de Recursos Humanitários (Opção 3)

Essa funcionalidade permite que o usuário localize **recursos humanitários próximos**. Para isso, o sistema consulta a **API Overpass** (mecanismo de consultas do OpenStreetMap) por meio de uma consulta QL construída dinamicamente, filtra resultados irrelevantes, ordena os resultados pela distância calculada com a **fórmula de Haversine** e apresenta, para cada local encontrado, seu **endereço**, **distância** e **número de telefone** quando disponíveis.

Os tipos de recursos suportados são:

- **hospital** (hospital)
- **shelter** (abrigo)
- **bus_station** (estação rodoviária/ponto de transporte)

A função **get_facility_name()** é responsável pela normalização das tags do OpenStreetMap, extraindo um nome legível independentemente do idioma utilizado nos dados. Dessa forma, o sistema consegue identificar corretamente o nome do local garantindo uma apresentação mais consistente e compreensível das informações ao usuário.

```
def get_facility_name(tags, amenity_type):
    if amenity_type == "shelter":
        return tags.get("shelter_type", "Emergency Shelter")
    elif amenity_type == "bus_station":
        return tags.get("name:en") or tags.get("name", "Bus Station")
    else:
        return tags.get("name:en") or tags.get("name", "Unknown facility")
```

4.3 Monitor de Alertas de Ataque Aéreo (Opção 4)

Esta funcionalidade consulta a **API Ukraine Alarm**. O sistema encontra todos os alertas ativos no momento em diferentes regiões da Ucrânia e exibe o nome de cada região juntamente com o tipo de alerta correspondente (por exemplo, **AIR** para ataque aéreo ou **ARTILLERY** para bombardeio de artilharia).

4.4 Autenticação com a ACLED (**get_token**)

O sistema realiza a autenticação na API da ACLED. As credenciais necessárias para a autenticação são carregadas a partir de um arquivo .env por meio da biblioteca python-dotenv, mantendo informações sensíveis fora do código-fonte e contribuindo para melhores práticas de segurança.

Após a autenticação bem-sucedida, a API retorna um token de acesso (access token), que é armazenado e reutilizado em todas as requisições subsequentes à ACLED durante a sessão da aplicação. Essa abordagem reduz a quantidade de solicitações de autenticação, melhora a eficiência do sistema e minimiza a exposição desnecessária de credenciais durante a execução do programa.

5. Conceitos de Python Aplicados

Os seguintes conceitos abordados durante o primeiro semestre foram aplicados ao longo do desenvolvimento do projeto:

Conceito	Aplicação no Projeto
Estruturas de Decisão (if/elif/else, match-case)	Utilização de match-case no despachante do menu principal; cadeia de if-elif na função risk_label() ; instruções if para filtragem de abrigos e seleção do tipo de recurso humanitário.
Estruturas de Repetição (while, for)	Laço while True responsável pelo funcionamento contínuo do menu principal; laços for utilizados para percorrer eventos de conflito, recursos humanitários e alertas ativos durante a exibição dos dados ao usuário.
Listas e Strings	Manipulação da lista events , incluindo ordenação e seleção dos três elementos mais próximos através de [:3] ; filtragem e ordenação da lista amenities ; utilização de f-strings para formatação de todas as saídas exibidas no terminal.
Funções e Procedimentos (def)	Encapsulamento da lógica do sistema em funções específicas, incluindo haversine() , troop_risk() , missile_strike() , risk_label() , get_token() , get_conflicts_events() , get_nearby_resources() , get_facility_name() e get_air_alerts() .
Passagem de Parâmetros e Valores de Retorno	As funções matemáticas e de integração com APIs recebem parâmetros de entrada e retornam valores processados. As funções de risco, por exemplo, recebem a distância (d) e a constante de decaimento (k), utilizando valores padrão quando apropriado.
Importação de Módulos (import)	Arquitetura modular composta pelos arquivos api.py , risk_model.py e support_functions.py , importados pelo arquivo principal main.py . Também foram utilizados módulos da biblioteca padrão, como math e os , além de bibliotecas externas como requests , aiohttp , asyncio e python-dotenv .